DS 5 d'informatique - MPSI - Graphes planaires - Correction

1. (a) Une graphe non orienté G est un couple $G = (S, A)$ où S est un ensemble fini et :

$$A \subset \Big\{\{x, y\} : (x, y) \in S^2\Big\}.$$

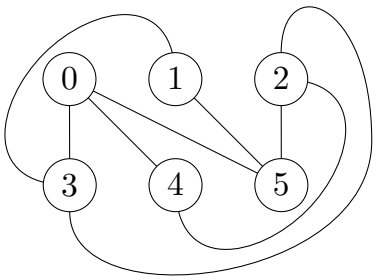
S s'appelle l'ensemble des sommets et A s'appelle l'ensemble des arêtes.

(b) Une boucle est une arête de la forme $\{s, s\}$ où $s \in S$.

2. Les deux listes sont identiques :

```
G_fig2 = [[1,2,3,4], [0,2,3,4], [0,1,3,4], [0,1,2], [0,1,2]]
G_fig3 = [[1,2,3,4], [0,2,3,4], [0,1,3,4], [0,1,2], [0,1,2]]
```

3. On dessine le graphe sans que les arêtes ne se croisent :



4.

```python
def suppr(L, i0):
    """
    L: list[int]
    i0: int
    Return: list[int]
    """
    M = []
    for i in L:
        if i != i0:
            M.append(i)
    return M
```

5.

```python
def decaler(L, i0):
    """
    L: list[int]
    i0: int
    Return: NoneType
    """
    for k in range(len(L)):
        if L[k] > i0:
            L[k] = L[k] - 1
    return None
```

6.

```python
def suppr_doublons(L):
    """
    L: list[int]
    Return: list[int]
    """
    d = {}  # Dictionnaire dont les clés sont les éléments de M
    M = []
    for i in L:
        if i not in d:
            d[i] = None
            M.append(i)
    return M
```

7. (a)

```python
def suppr_arete(G1, i1, i2):
    """
    G1: graphe
    i1, i2: int
    Return: graphe
    """
    G2 = []
    for i in range(len(G1)):
        if i == i1:
            G2.append(suppr(G1[i1], i2))
        elif i == i2:
            G2.append(suppr(G1[i2], i1))
        else:
            G2.append(G1[i][:])
    return G2
```

(b)

```python
def suppr_sommet(G1, i0):
    """
    G1: graphe
    i0: int
    Return: graphe
    """
    G2 = []
    for i in range(len(G1)):
        if i != i0:
            L = suppr(G1[i], i0)
            decaler(L, i0)
            G2.append(L)
    return G2
```

(c)

```python
def remplacer(L, j1, j2):
    """
    L: list[int]
    j1, j2: int
    Return: list[int]
    ------------------
    Remplace les j1 par j2 dans L
    """
    M = []
    for i in L:
        if i == j1:
            M.append(j2)
        else:
            M.append(i)
    return M
```

```python
def contr_arete(G1, i1, i2):
    """
    G1: graphe
    i1, i2: int
    Return: graphe
    """
    G2 = []
    j1 = min(i1,i2)
    j2 = max(i1,i2)
    for i in range(len(G1)):
        if i != j2:
            if i == j1:
                L1 = suppr(G1[j1], j2)
                L2 = suppr(G1[j2], j1)
                L = L1 + L2
            else:
                L = G1[i][:]
                L = remplacer(L, j2, j1)
            L = suppr_doublons(L)
            decaler(L, j2)
            G2.append(L)
    return G2
```

8. (a) Un graphe complet est un graphe dans lequel chaque sommet est voisin de tous les autres. Formellement :

$$A = \Big\{\{s,t\} : s \in S, t \in S, s \neq t\Big\}$$

(b)

```python
def make_complet(n):
    """
    n: int
    Return: graphe
    """
    G = []
    for i in range(n):
        G.append([])
        for j in range(n):
            if i != j:
                G[i].append(j)
    return G
```

9. (a) Puisqu'il n'y a pas de boucle et pas d'élément en double dans les listes d'adjacence, il suffit de vérifier que le graphe contient `n` sommets et que chaque liste d'adjacence est de taille `n-1`.

```python
def est_complet(G, n):
    """
    G: graphe
    n: int
    Return: bool
    """
    if len(G) != n:
        return False
    for L in G:
        if len(L) != n-1:
            return False
    return True
```

   (b) Toutes les opérations utilisées dans la fonction `est_complet` sont élémentaires et s'exécutent en temps constant. Il y a une boucle `for` qui exécute au plus `n` tours de boucle.
   En conclusion, le temps d'exécution est bien en $\mathcal{O}(\texttt{n})$.

10. Supposons par l'absurde que le graphe est biparti. On note S_1 et S_2 les ensembles associés.
Pour $i \in [\![0, 5]\!]$, on note s_i le sommet étiqueté par i. Supposons que $s_0 \in S_1$ (le cas où $s_0 \in S_2$ se traite de manière symétrique). Puisque s_2 et s_3 sont voisins de s_0, on a nécessairement $s_2 \in S_2$ et $s_3 \in S_3$. Ceci contredit la définition d'un graphe biparti puisque $\{s_2, s_3\}$ est une arête.

11.

```python
def make_biparti_complet(n1, n2):
    """
    n1, n2: int
    Return: graphe
    """
    G = []
    for i in range(n1):
        G.append([j for j in range(n1,n1+n2)])
    for i in range(n1,n1+n2):
        G.append([j for j in range(n1)])
    return G
```

12.

```python
import collections

def parc_larg(G, i0):
    """
    G: graphe
    i0: int
    Return: list[int]
    """
    V = []
    D = collections.deque([i0])
    vus = {i0: True}
    while len(D) > 0:
        s = D.popleft()
        V.append(s)
        for t in G[s]:
            if not t in vus:
                vus[t] = True
                D.append(t)
    return V
```

13.

```python
def parc_larg2(G):
    """
    G: graphe
    Return: list[int]
    """
    V = [False] * len(G)
    P = []
    for i in range(len(V)):
        if not V[i]:
            L = parc_larg(G, i)
            for j in L:
                V[j] = True
            P = P + L
    return P
```

14.

```python
def decomp_biparti(G):
    """
    G: graphe
    Return: (list[bool] * list[bool]) ou NoneType
    """
    n = len(G)
    S1 = [False] * n
    S2 = [False] * n
    P = parc_larg2(G)
    for i in range(n):
        if not S1[i] and not S2[i]:
            S1[i] = True
        if S1[i]:
            for j in G[i]:
                if S1[j]:
                    return None
                S2[j] = True
        else:
            for j in G[i]:
                if S2[j]:
                    return None
                S1[j] = True
    return S1, S2
```

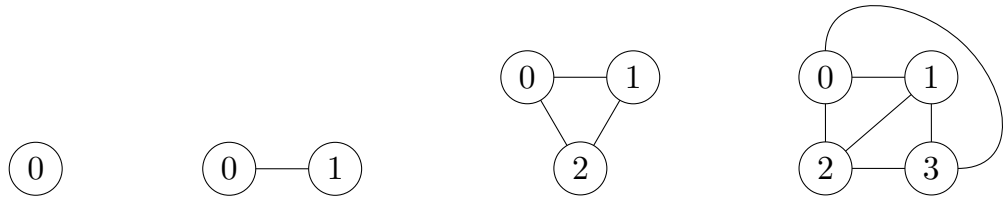
15.

```python
def est_biparti_complet(G, n1, n2):
    """
    G: graphe
    n1, n1: int
    Return bool
    """
    if len(G) != n1 + n2:
        return False
    S = decomp_biparti(G)
    if S is None:  # ou S == None
        return False
    S1, S2 = S
    T1 = [i for i in range(len(S1)) if S1[i]]
    T2 = [i for i in range(len(S2)) if S2[i]]
    m1 = len(T1)
    m2 = len(T2)
    if not ((n1,n2) == (m1,m2) or (n2,n1) == (m1,m2)):
        return False
    for i in T1:
        if len(G[i]) != m2:
            return False
    for i in T2:
        if len(G[i]) != m1:
            return False
    return True
```

16. (a) Montrons que le graphe G est planaire si et seulement si $n \leqslant 4$.

⋆ Lorsque $n = 0$, le graphe ne contient aucun sommet et est donc planaire.

⋆ Lorsque $1 \leqslant n \leqslant 4$, le graphe est planaire comme le montrent les schémas suivants :

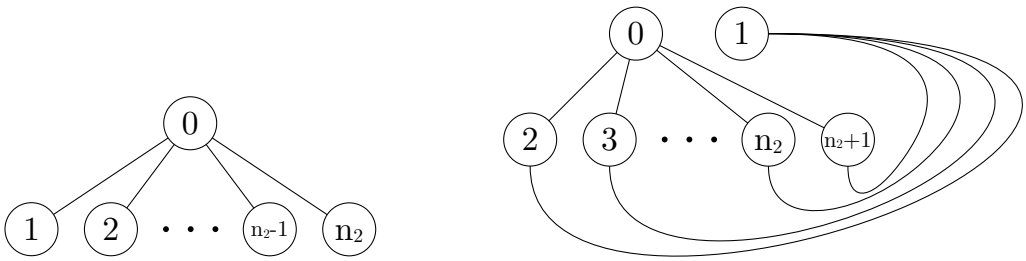


⋆ Lorsque $n > 4$, en supprimant $n - 5$ sommets, on obtient le graphe K_1. Par le théorème, le graphe n'est donc pas planaire.

(b) Montrons que le graphe G est planaire si et seulement si $n_1 \leqslant 2$ ou $n_2 \leqslant 2$.

⋆ Lorsque $n_1 = 0$, le graphe contient n_2 sommets mais aucune arête. Il est donc planaire.

⋆ Lorsque $n_1 = 1$ ou $n_1 = 2$, le graphe est planaire comme on peut le voir sur les schémas suivants :



⋆ Lorsque $n_2 = 0$ ou $n_2 = 1$ ou $n_2 = 2$, on montre comme dans les cas précédents que le graphe est planaire.

⋆ Sinon, on a $n_1 \geqslant 3$ et $n_2 \geqslant 3$. Si on supprime $n_1 - 3$ sommets de S_1 et $n_2 - 3$ sommets de S_2, on obtient le graphe K_2. Par le théorème, le graphe n'est donc pas planaire.

17. (a)

```
def mineurs_ordre1(G):
    """
    G: graphe
    Return: lsit[graphe]
    """
    L = []
    for i in range(len(G)):
        L.append(suppr_sommet(G, i))
        for j in G[i]:
            if i < j:
                L.append(suppr_arete(G, i, j))
                L.append(contr_arete(G, i, j))
    return L
```

(b)

```python
def est_planaire(G):
    """
    G: graphe
    Return: bool
    """
    if est_complet(G, 5):
        return False
    elif est_biparti_complet(G, 3, 3):
        return False
    L = mineurs_ordre1(G)
    for G2 in L:
        if not est_planaire(G2):
            return False
    return True
```